LAPORAN PRAKTIKUM FLUTTER
PERTEMUAN 4

MENGAMBIL DATA DARI REST API (HTTP, ASYNC, DAN FUTUREBUILDER)

Disusun Oleh:

Abdullah Wahid 20240801177

Dosen Pengampu:

JEFRY SUNUPURWA ASRI , S.Kom., M.Kom.

FAKULTAS ILMU KOMPUTER

UNIVERSITAS ESA UNGGUL

2026

BAB I — PENDAHULUAN

1.1 Latar Belakang

Aplikasi Flutter dapat mengambil data dari layanan web melalui REST API. Data yang diterima biasanya berbentuk JSON dan perlu diolah sebelum ditampilkan pada antarmuka. Pengambilan data berjalan secara asynchronous sehingga aplikasi perlu menangani kondisi menunggu, berhasil, dan gagal.

1.2 Tujuan Praktikum

- Memahami penggunaan Future dan FutureBuilder dalam Flutter.
- Mengambil data REST API menggunakan package http.
- Mengubah data JSON menjadi objek model melalui fromJson.
- Menampilkan daftar postingan, detail postingan, dan komentar.
- Menerapkan indikator loading, penanganan error, dan percobaan ulang.

BAB II — LANGKAH KERJA

Membuat proyek Flutter

Membuat proyek Flutter baru di Android Studio dengan nama daftar_postingan, lalu membuka file lib/main.dart.

Menambahkan package HTTP

Menambahkan package http pada bagian dependencies di pubspec.yaml, kemudian menjalankan Pub get.

Mengatur izin internet

Menambahkan izin INTERNET di AndroidManifest.xml, di dalam tag manifest dan sebelum tag application.

Membuat model data

Membuat model Post dan Komentar yang memiliki factory fromJson untuk membaca data JSON.

Membuat layanan API

Membuat ApiService untuk mengambil daftar postingan dari https://jsonplaceholder.typicode.com/posts dan komentar dari https://jsonplaceholder.typicode.com/posts/{id}/comments.

Membuat halaman daftar

Menampilkan judul dan cuplikan isi postingan dengan FutureBuilder. Halaman menyediakan indikator loading, pesan error, tombol coba lagi, refresh, dan keadaan data kosong.

Membuat halaman detail

Saat postingan dipilih, aplikasi menampilkan isi lengkap postingan dan mengambil komentar melalui Future terpisah.

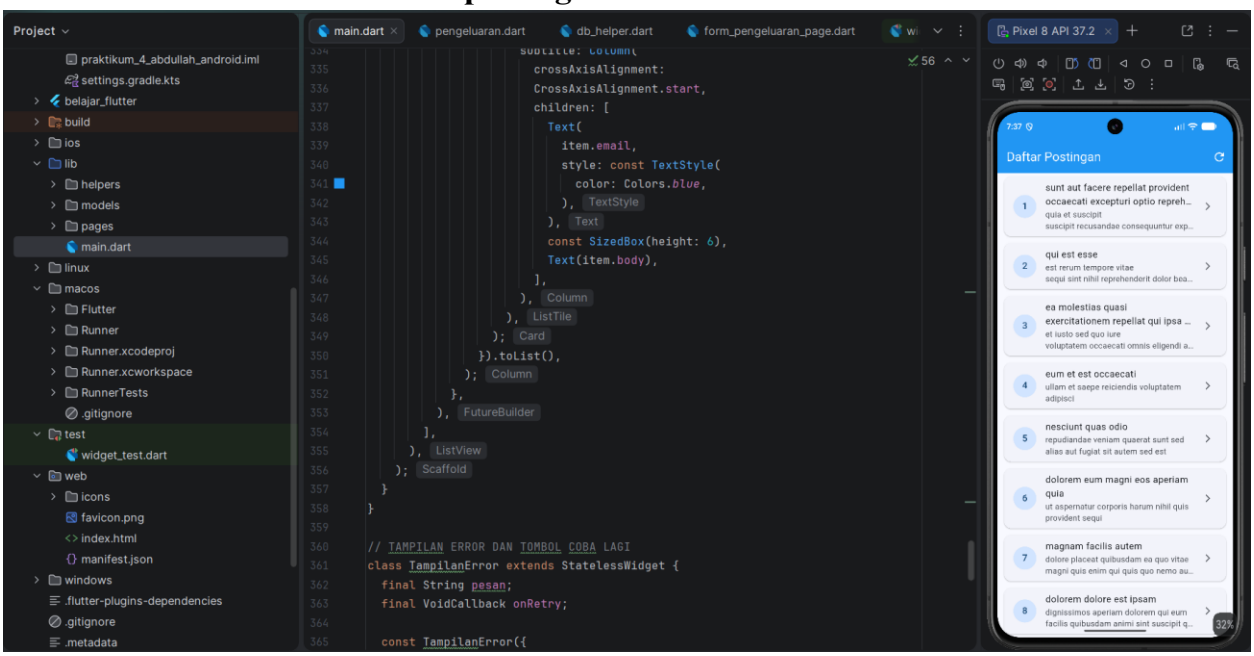
Menjalankan dan menguji aplikasi

Menjalankan aplikasi pada emulator atau perangkat Android, kemudian menguji daftar, detail, refresh, serta keadaan error.

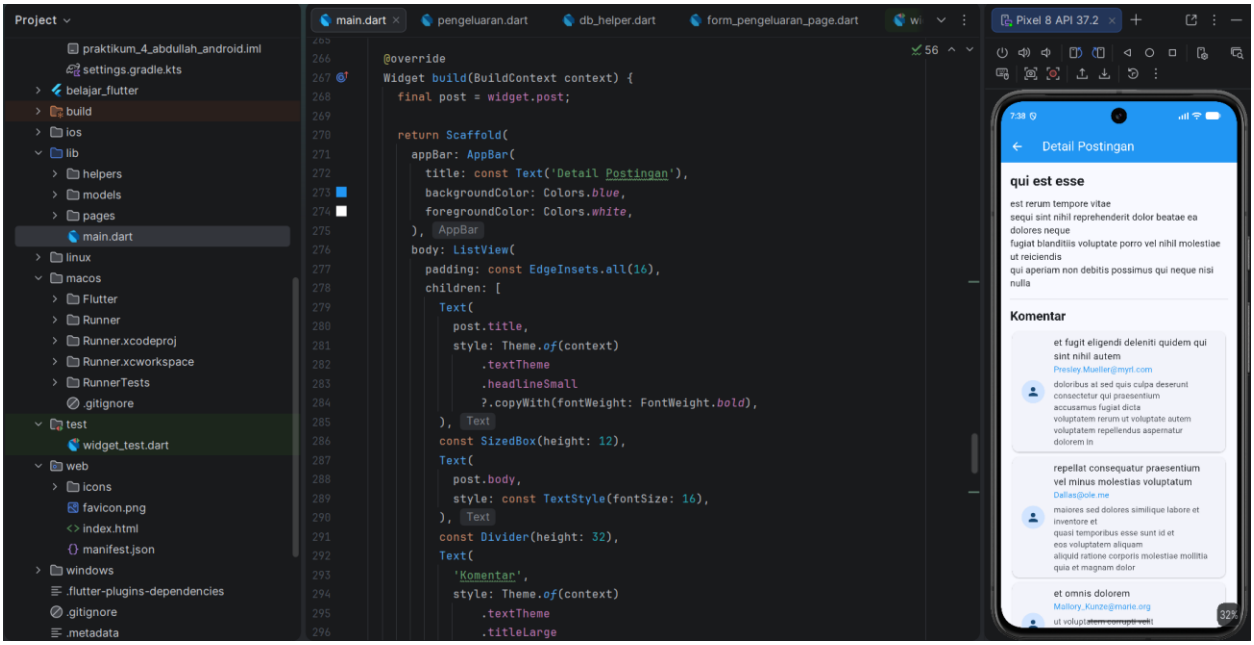
BAB III — HASIL DAN PEMBAHASAN

Lengkapi bagian ini dengan screenshot hasil aplikasi yang benar-benar dijalankan di Android Studio. Ganti petunjuk dalam tanda kurung siku dengan screenshot milik sendiri.

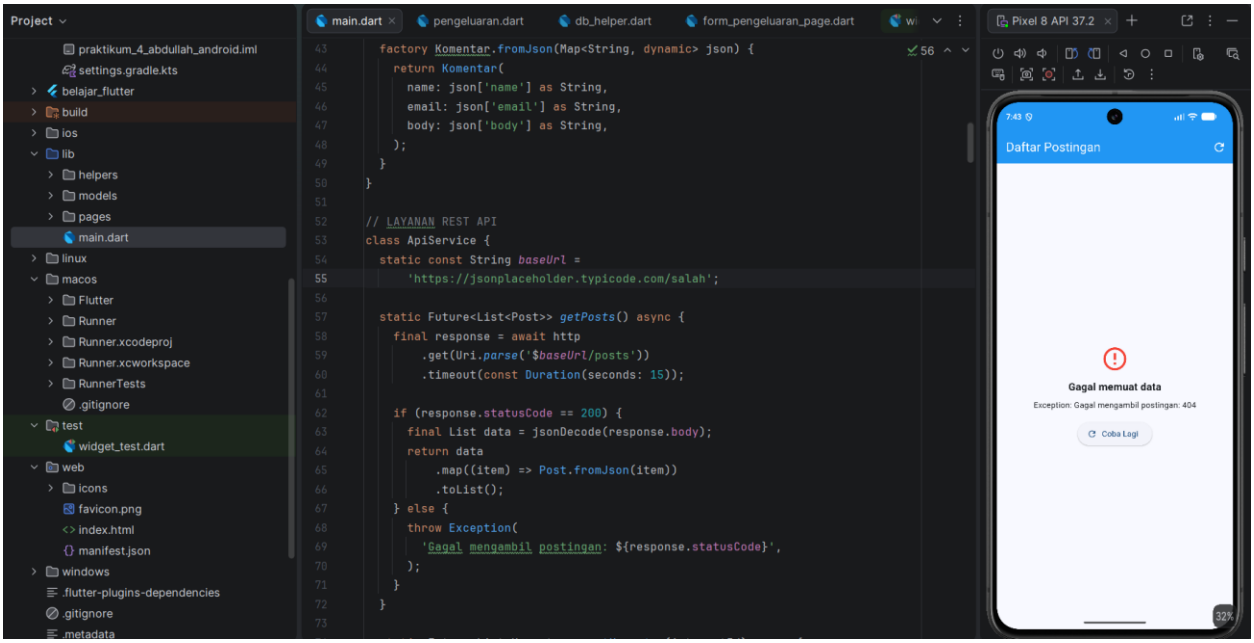
3.1 Screenshot halaman daftar postingan



3.2 Screenshot halaman detail dan komentar



3.3 Screenshot kondisi error



3.4 Screenshot kode program

```dart
// MODEL POSTINGAN
class Post {
  final int id;
  final String title;
  final String body;

  Post({
    required this.id,
    required this.title,
    required this.body,
  });

  factory Post.fromJson(Map<String, dynamic> json) {
    return Post(
      id: json['id'] as int,
      title: json['title'] as String,
      body: json['body'] as String,
    );
  }
}
```

```dart
// LAYANAN REST API
class ApiService {
  static const String baseUrl =
      'https://jsonplaceholder.typicode.com/salah';

  static Future<List<Post>> getPosts() async {
    final response = await http
        .get(Uri.parse('$baseUrl/posts'))
        .timeout(const Duration(seconds: 15));

    if (response.statusCode == 200) {
      final List data = jsonDecode(response.body);
      return data
          .map((item) => Post.fromJson(item))
          .toList();
    } else {
      throw Exception(
        'Gagal mengambil postingan: ${response.statusCode}',
      );
    }
  }
```

```
152        body: FutureBuilder<List<Post>>(
153          future: futurePosts,
154          builder: (context, snapshot) {
155            if (snapshot.connectionState ==
156                ConnectionState.waiting) {
157              return const Center(
158                child: CircularProgressIndicator(),
159              ); Center
160            }
161
162            if (snapshot.hasError) {
163              return TampilanError(
164                pesan: snapshot.error.toString(),
165                onRetry: muatUlang,
166              ); TampilanError
167            }
168
169            final posts = snapshot.data ?? [];
170
171            if (posts.isEmpty) {
172              return const Center(
173                child: Text('Tidak ada data'),
174              ); Center
175            }
176
177            return RefreshIndicator(
178              onRefresh: () async {
179                final future = ApiService.getPosts();
180                setState(() {
181                  futurePosts = future;
182                });
```

BAB IV — JAWABAN PERTANYAAN REFLEKSI

1. Mengapa pengambilan Future tidak sebaiknya dibuat berulang langsung di build?

Metode build dapat dipanggil berkali-kali ketika widget dibangun ulang. Jika Future dibuat langsung di build, permintaan API dapat terulang dan menggunakan sumber daya secara tidak perlu. Future dapat disimpan atau dibuat pada waktu yang sesuai, misalnya di initState, agar prosesnya lebih terkendali.

2. Apa perbedaan snapshot.hasError dan response.statusCode?

snapshot.hasError digunakan untuk mengetahui apakah proses asynchronous yang dikelola FutureBuilder menghasilkan error. response.statusCode adalah kode status HTTP dari server,

misalnya 200 untuk respons berhasil. Keduanya berbeda; respons HTTP gagal perlu diperiksa melalui statusCode dan dapat diubah menjadi exception agar ditangani sebagai error.

3. Mengapa menggunakan class model daripada Map secara langsung?

Class model seperti Post dan Komentar membuat struktur data lebih jelas, tipe data lebih mudah dikendalikan, dan akses properti lebih mudah dibaca. Factory fromJson memusatkan proses konversi JSON ke objek.

4. Mengapa aplikasi perlu menampilkan loading dan error?

Pengambilan data dari internet membutuhkan waktu dan dapat gagal karena jaringan atau respons server. Indikator loading memberi tahu pengguna bahwa proses sedang berlangsung, sedangkan pesan error dan tombol coba lagi membantu pengguna memahami masalah dan mengulangi permintaan.

BAB V

KESIMPULAN

Melalui praktikum ini, penggunaan REST API pada Flutter dapat dipahami dengan memanfaatkan package http untuk mengambil data, model fromJson untuk mengolah JSON, serta FutureBuilder untuk menampilkan proses asynchronous. Aplikasi Daftar Postingan menampilkan daftar, detail, dan komentar, serta perlu menangani loading, error, dan percobaan ulang agar lebih mudah digunakan.